Backend README: User CRUD API

A simple REST API for managing users. Built for learning.

Tech Stack

- **Node.js**: runs JavaScript on the server.
- **Express**: web framework that defines the API routes.
- **TypeScript**: JavaScript with types, run in development by `tsx`.
- **Prisma 7**: ORM. You describe tables in a schema file and call typed functions like `findMany` and `create` instead of writing SQL by hand.
- **SQLite**: a database stored in a single file (`dev.db`). No server to install.
- **Driver adapter (`@prisma/adapter-better-sqlite3`)**: Prisma 7 needs an adapter, a small bridge between Prisma and the actual database. Flow: your code -> Prisma -> adapter -> SQLite file.
- **cors**: lets the frontend (port 3000) call the backend (port 3001).
- **dotenv**: loads settings from the `.env` file.

Folder Structure

```
backend/
  prisma/
    schema.prisma      <- table definitions (the User model)
    migrations/        <- saved SQL history
  generated/prisma/    <- generated Prisma client (do not edit)
  src/
    index.ts           <- Express server and all routes
  .env                 <- DATABASE_URL and PORT
  prisma7.config.ts    <- Prisma CLI config
  package.json
  tsconfig.json
```

Setup From Scratch

1. Create the folder and initialize: `mkdir backend`, `cd backend`, `npm init -y`.

2. Install runtime packages: `npm install express cors dotenv @prisma/client @prisma/adapter-better-sqlite3`.
3. Install dev packages: `npm install -D typescript tsx prisma @types/node @types/express @types/cors`.
4. Create the TypeScript config: `npx tsc --init`.
5. Initialize Prisma: `npx prisma init --datasource-provider sqlite`. This creates `prisma/schema.prisma`, `prisma7.config.ts`, and `.env`.

Config Fixes Needed

Two errors appear right after setup. Both are config issues, not bugs.

- **Import error in `prisma7.config.ts`**: add `"type": "module",` to `package.json`. This tells Node the project uses modern `import` syntax.
- **Cannot find name `process`**: in `tsconfig.json` set `"types": ["node"],` so TypeScript knows Node's built-ins.

Also confirm in `tsconfig.json`: `"module": "nodenext"`, `"target": "esnext"`, `"moduleResolution": "nodenext"`.

Environment Variables (`.env`)

```
DATABASE_URL="file:./dev.db"
PORT=3001
```

Use port 3001 for the backend because Next.js uses 3000.

Prisma Schema

Keep the generated `generator` and `datasource` blocks, and add this model at the bottom of `prisma/schema.prisma`:

```
model User {
  id        Int      @id @default(autoincrement())
  name      String
  email     String   @unique
  createdAt DateTime @default(now())
}
```

- `model User` becomes the User table.
- `@id @default(autoincrement())` makes the primary key count 1, 2, 3 automatically.
- `@unique` blocks duplicate emails.
- `@default(now())` fills in the creation time.

Create the Database

```
npx prisma migrate dev --name init
npx prisma generate
```

- `migrate dev` creates `dev.db`, writes a SQL migration file under `prisma/migrations`, and runs it.
- `generate` creates the typed client in `generated/prisma`. Run it again after every schema change.
- Optional: `npx prisma studio` opens a browser view of your data.

The Server (`src/index.ts`)

```
import "dotenv/config";
import express from "express";
import cors from "cors";
import { PrismaBetterSqlite3 } from "@prisma/adapter-better-sqlite3";
import { PrismaClient } from "../generated/prisma/client.js";

const adapter = new PrismaBetterSqlite3({ url: process.env.DATABASE_URL! });
const prisma = new PrismaClient({ adapter });

const app = express();
app.use(cors());
app.use(express.json());

app.get("/users", async (_req, res) => {
  const users = await prisma.user.findMany();
  res.json(users);
});

app.get("/users/:id", async (req, res) => {
  const user = await prisma.user.findUnique({
    where: { id: Number(req.params.id) },
```

```
  });
  if (!user) return res.status(404).json({ error: "User not found" });
  res.json(user);
});

app.post("/users", async (req, res) => {
  const { name, email } = req.body;
  if (!name || !email) {
    return res.status(400).json({ error: "Name and email are required" });
  }
  try {
    const user = await prisma.user.create({ data: { name, email } });
    res.status(201).json(user);
  } catch {
    res.status(409).json({ error: "Email already exists" });
  }
});

app.put("/users/:id", async (req, res) => {
  const { name, email } = req.body;
  try {
    const user = await prisma.user.update({
      where: { id: Number(req.params.id) },
      data: { name, email },
    });
    res.json(user);
  } catch {
    res.status(404).json({ error: "User not found or email already exists"
});
  }
});

app.delete("/users/:id", async (req, res) => {
  try {
    await prisma.user.delete({ where: { id: Number(req.params.id) } });
    res.status(204).send();
  } catch {
    res.status(404).json({ error: "User not found" });
  }
});
```

```
const PORT = process.env.PORT || 3001;
app.listen(PORT, () => console.log(`Server running on
http://localhost:${PORT}`));
```

Keep `app.listen` last, so the server starts after all routes are defined.

Run It

Add to `package.json` scripts: `"dev": "tsx watch src/index.ts"`. Then run `npm run dev`. The server restarts automatically on every save.

API Endpoints

Method	Endpoint	Description	Success status
GET	`/users`	List all users	200
GET	`/users/:id`	Get one user	200 (404 if missing)
POST	`/users`	Create a user (body: name, email)	201 (409 if email exists)
PUT	`/users/:id`	Update a user (body: name, email)	200
DELETE	`/users/:id`	Delete a user	204

Testing With Postman

1. GET `http://localhost:3001/users` returns an empty array.
2. POST the same address: Body tab, choose raw, change Text to JSON, send an object with `name` and `email`.
3. GET again to see the new user, then GET `/users/1`.
4. PUT `/users/1` with a changed name.
5. DELETE `/users/1` returns 204.

Key Concepts Learned

- **ORM**: maps tables to models, rows to objects, columns to properties. Prisma plays the role Mongoose plays for MongoDB.
- **Migration**: a saved, versioned step in your database history.
- **Adapter**: the bridge between Prisma and a specific database.
- **SQLite**: a real database in one file. It works on a server with a persistent disk (VPS, EC2, Lightsail), but not on hosts that wipe the disk, such as Render's free tier or AWS Lambda.
- **`DATABASE_URL`**: the database address, set as an environment variable so the same code runs everywhere. For SQLite it is a file path. For PostgreSQL it looks like `postgresql://user:password@host:5432/dbname`.
- **Managed databases**: Neon, Supabase, and AWS RDS host PostgreSQL for you, and MongoDB Atlas does the same for MongoDB. A VPS is a blank rented computer where you install everything yourself.

Next Steps

1. Switch to PostgreSQL: run Postgres in Docker, set the datasource provider to `postgresql`, swap in the Postgres adapter, update `DATABASE_URL`, and migrate again.
2. Put the Express app in a Docker container and use Docker Compose to run the backend and Postgres together.
3. Deploy with the database on Neon or Supabase.